

Socratic Prompting:

I want to build a scalable, data-driven Selenium automation framework.

Do not provide the implementation immediately. Instead, act as a Solution Architect and gather my requirements using the Socratic questioning technique.

Rules:

- * Ask exactly one question at a time.
- * Each question should provide multiple-choice options (A, B, C, D...) along with a brief explanation of each option.
- * Wait for my response before asking the next question.
- * Ask only the questions necessary to fully understand my requirements.
- * Do not make assumptions unless I explicitly ask for recommendations.

Once all the requirements are gathered:

1. Summarize my chosen requirements.
2. Explain the architecture and design decisions.
3. Provide the recommended project structure.
4. Implement the complete framework step by step with well-commented Java code.
5. Explain why each design choice was made and discuss possible alternatives.
6. Follow industry best practices for Selenium, TestNG, Maven, Page Object Model, and data-driven testing.

Result:

1. Your confirmed requirements

Area	Your choice
Application	Web application
Browser	Single browser
Primary testing	Functional UI testing
Language	Java 17
Automation	Selenium
Test framework	TestNG
Build	Maven
Design pattern	Page Object Model
Business workflows	Test classes directly interact with Page Objects
Test data	Excel
Excel organization	One Excel file per module/feature
Excel location	src/test/resources/testdata
Excel sheet	Fixed TestData sheet
Excel structure	Header row + dataset rows
Test identification	Test Case ID
Dataset identification	Dataset ID
Duplicate Dataset IDs	Allowed
Excel values	Convert everything to String
Empty cells	Empty string
Blank TestCaseID rows	Ignore
Excel validation	Validate structure when reading

Area	Your choice
Excel errors	Fail fast
DataProvider	Reusable Excel utility + TestNG DataProvider
Multiple datasets	All matching datasets executed
Test data execution	TestCaseID + DatasetID
POM	Page Objects
Locators	Directly inside Page Objects
Waits	Explicit waits + reusable WaitUtility
Dynamic elements	Robust Selenium locators + explicit waits
Driver management	ThreadLocal<WebDriver> + DriverManager
Driver cleanup	@AfterMethod, always quit
Browser configuration	Maven -Dbrowser
Application URL	Maven -Durl
Environment	Single environment
Configuration	Maven runtime parameters
Test organization	One test class per feature/module
Test independence	Tests completely independent
Assertions	TestNG assertions directly in tests
Reporting	Allure
Failure evidence	Screenshot on failure + important steps
Retry	No retry
Logging	SLF4J + Log4j2
CI/CD	Azure DevOps
Pipeline configuration	Azure DevOps variables

Area	Your choice
Maven execution	Surefire + TestNG
Selenium version	Latest stable version at implementation time
Failure handling	Clear failures rather than retries
Application unavailable	Test fails with clear error
Browser options	Minimal configuration
Selenium driver management	Maven-managed Selenium dependency
Shared test data	Duplicate within module files
Test setup	@BeforeMethod / @AfterMethod
Test naming	Descriptive business-oriented names

One important architectural consequence

You selected **ThreadLocal<WebDriver>**, even though you're initially running a single browser.

That's a good architectural choice for scalability: we can keep the implementation thread-safe without actually enabling parallel execution yet.

2. Proposed architecture

The framework will follow this flow

TestNG Test

|



Test Class

|



Page Object

|



DriverManager



ThreadLocal<WebDriver>



Selenium WebDriver



COMPASS UI

And for data:

Excel



ExcelReader



DataProvider



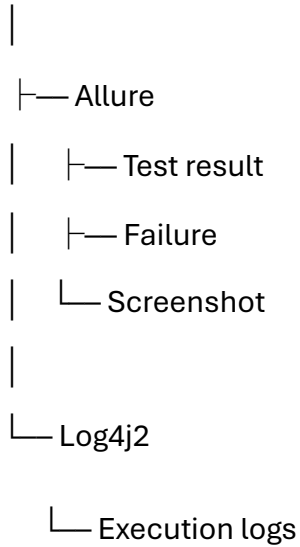
Test Method



Test Execution

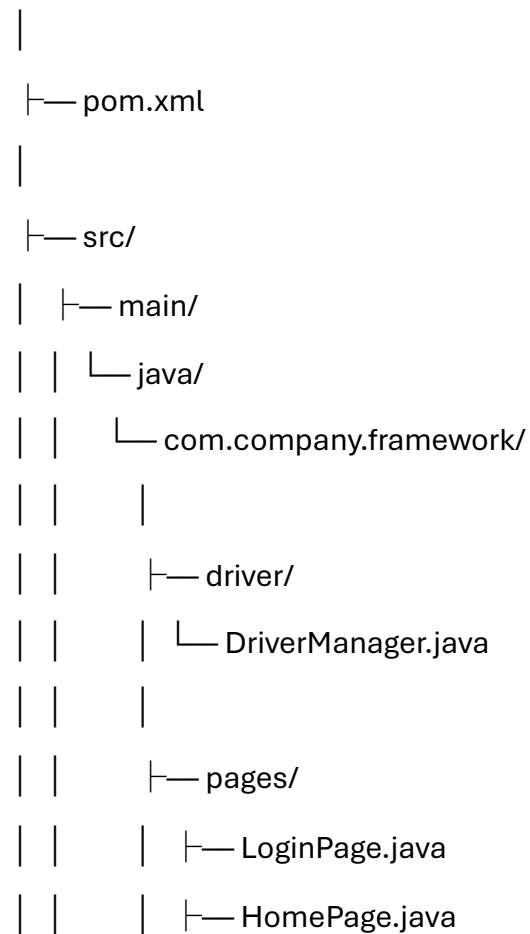
Reporting:

TestNG



3. Recommended project structure

selenium-automation-framework/



```
| | | |└─ PortfolioPage.java
| | | |
| | | |└─ utils/
| | | |
| | | |└─ ExcelReader.java
| | | |
| | | |└─ WaitUtility.java
| | | |
| | | |└─ AlertUtility.java
| | | |
| | | |
| | | |└─ listeners/
| | | |
| | | |└─ TestListener.java
| | | |
| | | |
| | | |└─ test/
| | | |
| | | |└─ java/
| | | |
| | | |└─ com.company.tests/
| | | |
| | | |
| | | |└─ base/
| | | |
| | | |└─ BaseTest.java
| | | |
| | | |
| | | |└─ login/
| | | |
| | | |└─ LoginTest.java
| | | |
| | | |
| | | |└─ portfolio/
| | | |
| | | |└─ PortfolioTest.java
| | | |
| | | |
| | | |└─ resources/
| | | |
| | | |
| | | |└─ testdata/
| | | |
| | | |└─ Login.xlsx
| | | |
| | | |└─ Portfolio.xlsx
```

[\[Open\]](#)

```
|      |  
|      └─ log4j2.xml  
|  
└─ azure-pipelines.yml
```

One thing we will **not** introduce unnecessarily is a large number of framework layers.
Your choices favor a clean, practical Selenium framework rather than over-engineering.